

ARES: Dataflow Co-Design for Embedded Swin Transformer Acceleration on SoC-FPGAs

João Barreiros Rodrigues
INESC-ID and INESC INOV
IST / Universidade de Lisboa
Lisbon, Portugal
jbrcr@inov.pt

Horácio C. Neto
INESC-ID and INESC INOV
IST / Universidade de Lisboa
Lisbon, Portugal
horacio.neto@inov.pt

Mário Véstias
INESC-ID and INESC INOV
ISEL / Instituto Politécnico de Lisboa
Lisbon, Portugal
mario.vestias@inov.pt

Abstract

Vision Transformers (ViTs) have demonstrated strong performance in computer vision by modeling long-range spatial dependencies through self-attention. Swin Transformers improve their scalability by introducing window-based attention and hierarchical representations, making them suitable for dense prediction tasks such as image segmentation. However, their computational and memory requirements hinder efficient deployment on resource-constrained embedded platforms.

Aerial image segmentation provides a demanding benchmark, combining high-resolution inputs with fine-grained spatial reasoning. This work presents a hardware–software co-designed Swin Transformer layer acceleration framework targeting FPGA-based edge devices. A Swin-based aerial segmentation model is profiled to identify dominant computational kernels, guiding a selective acceleration strategy.

Dataflow-oriented hardware accelerators are developed for the main transformer components, namely QKV projections, attention mechanisms, and the Feed-Forward Network (FFN). The FFN accelerator further integrates hardware-friendly activation replacement within the compute pipeline to reduce memory transfers and improve parallelism.

The accelerators are integrated into a complete embedded system, denoted ARES, and deployed on a ZCU104 SoC-FPGA platform. The implementation achieves up to 206× speedup for the FFN and 10.14× end-to-end acceleration of a Swin Transformer layer compared to ARM-only execution.

These results demonstrate the feasibility of deploying transformer-based segmentation models on embedded FPGAs under strict resource and latency constraints.

CCS Concepts

• **Computer systems organization** → **Embedded hardware**; *Embedded software*; • **Hardware** → **Hardware accelerators**; **Hardware–software codesign**; *Datapath optimization*; *Arithmetic and datapath circuits*.

Keywords

Vision Transformer, Swin Transformer, Aerial Image Segmentation, SoC-FPGA, Embedded Systems, Hardware Acceleration

ACM Reference Format:

João Barreiros Rodrigues, Horácio C. Neto, and Mário Véstias. 2026. ARES: Dataflow Co-Design for Embedded Swin Transformer Acceleration on SoC-FPGAs. In *Proceedings of the 16th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART 2026)*, June 17–19, 2026, Heidelberg, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3814576.3814594>

1 Introduction

Vision Transformers (ViTs) have become an important paradigm in computer vision, showing strong performance across many tasks by using self-attention to model long-range spatial dependencies. ViTs capture global context directly, which can improve representation learning, especially in complex or high-resolution images. However, this comes at the cost of substantial computation and memory requirements, making efficient deployment difficult, particularly on edge and embedded systems.

Swin Transformers address some of these limitations by replacing full global attention with shifted window attention. This design improves computational efficiency while still allowing information to flow across image regions, and introduces a hierarchical structure that better supports tasks such as detection and segmentation.

Despite these architectural advances, implementing Vision Transformers (ViTs) and their variants on resource-constrained platforms remains challenging due to their computational intensity and memory footprint. Offloading inference to cloud infrastructures is a common workaround, but it introduces additional latency, bandwidth overhead, and data-security concerns that are often unacceptable for mission-critical or real-time operations [19, 21]. In this context, System-on-Chip Field Programmable Gate Arrays (SoC-FPGAs) provide an effective solution for edge computing by combining domain-specific reconfigurable hardware accelerators with general-purpose embedded processors while maintaining energy efficiency [2].

This work focuses on the hardware–software co-design of a Swin Transformer layer accelerator and on its evaluation for efficient aerial image segmentation on embedded platforms. Aerial image segmentation models are well suited to evaluate both the accuracy and efficiency of the accelerator, since they combine high-resolution inputs with the need for precise spatial reasoning over large scenes. Applications such as urban planning, environmental monitoring, and disaster response demand efficient and reliable inference under strict latency and resource constraints.

A representative aerial image segmentation Swin-based model, AerialFormer [26], is adopted as a benchmark, and its execution is profiled to identify its most computationally intensive components, guiding the targeted acceleration strategy. The key components,



This work is licensed under a Creative Commons Attribution 4.0 International License. HEART 2026, Heidelberg, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2452-7/2026/06
<https://doi.org/10.1145/3814576.3814594>

namely the Feed-Forward Network (FFN), QKV generator, and attention blocks, are optimized and hardware-accelerated to improve performance under resource constraints. To further improve hardware acceleration efficiency, complex nonlinear operations such as GELU [5] and LayerNorm (LN) [1] are replaced with more hardware-friendly alternatives, reducing arithmetic complexity without significantly affecting model accuracy.

A modular inference framework is developed in C for bare-metal execution, serving as both a software baseline and a control layer for the hardware accelerators. The hardware components are designed using High-Level Synthesis (HLS), enabling rapid development [13, 17] and seamless integration into the complete system.

The resulting hardware–software system, denoted ARES, is implemented and validated on a ZCU104 platform featuring a XCZU7EV Zynq UltraScale+ SoC-FPGA, demonstrating significant performance improvements for transformer-based inference on embedded devices.

Contributions. This work bridges the gap between transformer-based vision models and efficient embedded deployment through a hardware–software co-design approach. The main contributions are:

- **Hardware-aware reformulation of the Swin Transformer layer.** We introduce a modified Swin Transformer formulation tailored for efficient hardware execution, including the replacement of hardware expensive operations (GELU and LN) with hardware-friendly alternatives. These modifications reduce arithmetic complexity while preserving model accuracy.
- **Quantized transformer design space exploration.** We perform a systematic evaluation of accuracy–efficiency trade-offs under reduced precision, analyzing the impact of quantization and architectural modifications on aerial image segmentation performance and providing practical guidelines for deploying transformer models under resource constraints.
- **Parameterized parallel architecture for transformer primitives.** We design a scalable architecture based on processing element (PE) arrays and streaming interfaces, supporting efficient implementation of key transformer operations. The architecture enables flexible trade-offs between resource utilization and performance.
- **Dataflow-oriented FFN accelerator.** We propose a novel Feed-Forward Network (FFN) accelerator that integrates activation function computation with two PE arrays within a streaming dataflow: operations execute as soon as data is available, maximizing inherent parallelism and minimizing intermediate memory transfers, to achieve high throughput.
- **Full system implementation and evaluation (ARES).** We implement the proposed approach on a ZCU104 platform featuring a XCZU7EV Zynq UltraScale+ SoC-FPGA. The resulting system achieves up to 206× speedup for the FFN and 10.14× end-to-end acceleration of a Swin Transformer layer, demonstrating the practicality of transformer deployment on embedded devices.

2 Background and Related Work

Transformers are a new Deep Learning Architecture proposed in [24] for natural language processing. They improve architectures such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs), by handling input sequence data in parallel and by efficiently capturing long-term dependencies between input segments, which the older architectures struggle with.

A generic Transformer architecture, as depicted in Figure 1, consists of an input representation pre-processor, followed by a stack of encoder layers and a decoder. Each encoder layer generates three parameters from its input: Query (Q), Key (K) and Value (V), integrates a Multi-Head Self-Attention (MHSA) mechanism [24] that operates over them and where SoftMax (SMax) is applied to the intermediary QK matrix, and a Projection layer, which is implemented as a General Matrix Multiplication (GEMM). Each encoder layer includes also a Feed-Forward Network (FFN), comprised of two linear transformations, implemented as GEMMs, sandwiching a GELU activation, and ensures rich features extraction without recurrent processing. Layer normalization and residual skip connections further stabilize training while preserving feature gradients throughout the stack.

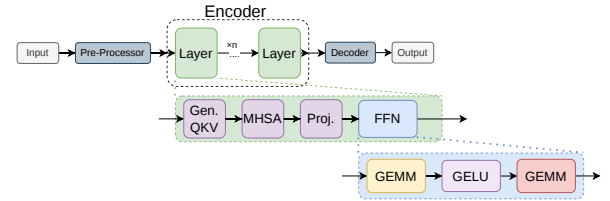


Figure 1: Generic Transformer Architecture, with normalization and skip connections omitted

As the attention mechanism models global, long-range relationships across an input, their architecture was applied to vision tasks, where the context of the complete image defines object classification.

Google’s ViT [4] first demonstrated competitive or superior performance to traditional Convolutional Neural Networks (CNNs) across various vision benchmarks, including image classification, object detection, and image segmentation. In Google’s ViT the input is segmented into coarse 16×16 patches by the input representation pre-processor, to limit the quadratic computational complexity of their Attention mechanism with respect to the total number of patches.

The Swin Transformer [14] is a vision transformer widely used for image segmentation, presented in late 2021 by Microsoft, designed with an efficient image representation focus and scalable performance. The Swin transformer was the first architecture to propose and employ Window Self-Attention (WSA), in its Transformer layers. In WSA the patches are grouped into windows, with the MHSA being calculated locally, intra-window. By implementing local WSA, the Swin Transformer is able to use smaller input image patches without increasing the computational complexity.

The Swin architecture is composed by a stack of 4 stages that operate at different spatial dimensions as shown in Figure 2. Each

of the Swin stages is composed of two or more Swin Layers. The number of layers in each stage depends on the specific Swin variant. The smallest Swin Transformer Architecture is the Swin-T (Tiny), which has 2, 2, 6 and 2 layers in its respective transformer stages. As the local WSA cannot directly gather global context, a shift operator is inserted between the Swin layers. Thus, each Swin layer output is shifted, changing the window location by half of the window size, to gradually build global context by integrating information from different windows.

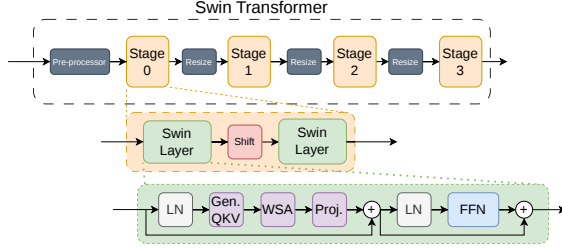


Figure 2: Swin Transformer Architecture

2.1 AerialFormer

AerialFormer [26] is a transformer-based architecture specifically designed for aerial image segmentation. It extends the Swin Transformer by incorporating a parallel CNN stem to preserve fine-grained spatial details that may be lost during patch embedding, as well as a Multi-Dilated Convolutional Neural Network (MDCNN) decoder to aggregate multi-scale features extracted by the transformer backbone. Combined with the use of small 4×4 patches, these design choices enable accurate segmentation of small and densely distributed objects, which are common in aerial imagery.

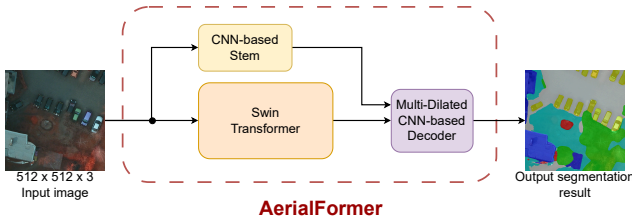


Figure 3: AerialFormer Model

Three variants of AerialFormer are provided, differing in model capacity and computational cost. Their complexity and performance on the Potsdam [9, 23] dataset are summarized in Tables 1a and 1b. Performance is evaluated using standard semantic segmentation metrics. We report mean Intersection over Union (mIoU) as the primary metric, complemented by overall accuracy (OA) to provide a global measure of classification performance.

The Potsdam dataset is a widely used benchmark for high-resolution aerial image segmentation, consisting of very high spatial resolution (5 cm/pixel) orthorectified images with detailed pixel-level annotations. It includes multiple urban classes such as buildings,

Table 1: AerialFormer (AF) variants and segmentation performance in the Potsdam dataset with clutter class, adapted from [26]

(a) Model complexity

	Params. (M)	FLOPs (G)
AF-T	42.7	49.0
AF-S	64.0	72.2
AF-B	113.8	126.8

(b) Segmentation performance (Potsdam)

	OA (%)	mIoU (%)
AF-T	91.1	79.5
AF-S	91.3	79.3
AF-B	91.4	79.7

roads, vegetation, and clutter, and is particularly challenging due to the presence of small objects, fine boundaries, and high intra-class variability. These characteristics make it well-suited for evaluating both segmentation accuracy and the ability of models to preserve spatial detail.

As shown in Table 1b, the three AerialFormer variants achieve comparable segmentation performance, with differences below 0.5% across all metrics. However, their computational requirements differ significantly, with the Tiny variant (AF-T) reducing the number of parameters and FLOPs by more than 2× compared to the Base variant.

Given the marginal accuracy gap and the substantial reduction in computational complexity, the AerialFormer-Tiny (AF-T) model is selected as the benchmark in this work. This choice is particularly motivated by the target deployment on resource-constrained embedded platforms, where model size, memory footprint, and computational efficiency are critical constraints. By focusing on the smallest variant that preserves competitive accuracy, this work enables a more meaningful evaluation of hardware–software co-design strategies under realistic edge deployment conditions.

2.2 FPGA implementations of Vision Transformers

Despite the rapid adoption of Vision Transformers in computer vision, relatively few FPGA-based accelerators have been reported, particularly for embedded platforms [12].

ViTA [16] proposes an embedded-oriented inference accelerator using the Google’s ViT-B model, implemented on a Zynq-7020 SoC-FPGA. The design employs 8-bit post-training quantization with negligible accuracy degradation ($< 0.04\%$ on ImageNet). Its heterogeneous architecture integrates dedicated processing elements (PEs) for QKV projection, attention computation, and FFN execution. Nonlinear operators, including SMax, LN, and GELU, are implemented using LUT-based approximations [8]. While LUT-based designs offer low-latency evaluation for nonlinear functions, they become resource-intensive as model dimensionality or quantization precision increases. Consequently, scalability to larger transformer variants or higher bitwidths are constrained on resource-limited

devices. ViTA primarily focuses on accelerating the standard ViT architecture, which differs structurally from hierarchical transformers such as Swin.

More recently, SWAT [3] targets Swin Transformer acceleration on data-center-class FPGAs, focusing on throughput optimization via tiling and pipeline interleaving techniques. Implemented on an Alveo U50 platform, SWAT emphasizes high computational density and large-scale parallelism. Although achieving substantial throughput improvements, its deployment model and resource budget differ significantly from edge-oriented SoC-FPGA systems.

Liu et al. [15] propose an FPGA-based accelerator specifically for Swin Transformer inference, introducing hardware-specific normalization and attention optimizations. Their approach also employs specialized functional units for calculating polynomial approximations to the Transformer’s nonlinear operations. These units are however resource costly, with the GELU using 98 Digital Signal Processors (DSPs) and over 50,000 Look-Up Tables (LUTs). Overall their work explores architectural acceleration of Swin components rather than system-level hardware–software co-design in an embedded context.

Overall, prior FPGA-based accelerators for Vision Transformers either target resource-rich platforms that enable full model acceleration or do not explicitly address embedded system constraints and hardware–software partitioning trade-offs. In contrast, this work investigates the practical deployment of a quantized Swin Transformer layer on a mid-range SoC-FPGA, focusing on targeted acceleration and resource-aware design.

2.3 Hardware-Oriented replacements of non-linear Transformer Components

The HardSwish (HSwish) function (detailed in Equation 1) was introduced in [6] as part of the MobileNetV3 architecture. It is a computationally efficient, piecewise-linear approximation of the Swish activation function, designed specifically for resource-constrained environments such as mobile and embedded systems. HSwish replaces the expensive sigmoid function with simple arithmetic and bounded linear components. This makes it particularly well-suited for hardware implementations, where it can be efficiently realized using additions, multiplications, and clipping operations.

$$\text{Hardswish}(x) = x \cdot \frac{\text{ReLU}(x + 3)}{6} \quad (1)$$

HSwish is widely used as a hardware-friendly alternative to Swish or ReLU-like activations for mobile/edge CNNs, but, to the best of our knowledge, it has not been reported as a replacement for GELU in transformer-style models.

SquareMax (SqrMax) [7] is a normalization function that replaces the exponential operation in SMax with a squared activation, as detailed in Equation 2. By relying only on ReLU and multiplication, it significantly reduces computational complexity while preserving the probabilistic interpretation of the output.

$$\text{SquareMax}(x_i) = \frac{\text{ReLU}(x_i)^2}{\sum_j \text{ReLU}(x_j)^2 + \epsilon} \quad (2)$$

The SqrMax function was explicitly designed as a hardware-efficient alternative to SMax, prioritizing computational simplicity and suitability for low-resource implementations.

Layer Normalization (LN) normalizes the features of each individual input sample by computing the mean and variance across its feature dimensions. This operation requires divisions and square-root computations, leading to relatively high hardware complexity.

Batch Normalization (BN) can be considered as a hardware-friendly alternative to LN. During inference, the running mean and variance estimated during training are fixed, allowing the normalization to be expressed as a linear transformation with pre-computed coefficients. Two main strategies have been proposed to integrate BN into transformer architectures [20]: Simple-BN and FFN-BN.

Simple-BN consists of directly replacing all LN layers with BN, followed by full model retraining. However, depending on the transformer architecture, this approach may negatively impact training stability and convergence.

To address this limitation, FFN-BN introduces additional BN layers after each fully connected layer within the FFN, as proposed in [27]. This strategy improves training behavior while enabling the use of BN in place of LN.

Due to the presence of residual (skip) connections in transformer architectures, BN layers cannot be trivially fused into preceding linear transformations, unlike in conventional feedforward neural networks deployed on FPGAs [11]. To maintain efficient hardware implementation, BN can instead be mapped to a 1×1 convolution with an added bias term, preserving its functionality while enabling efficient integration into the compute pipeline [22].

3 Model Optimization and Analysis

This section presents the methodology adopted to validate the optimized version of the model with some of the replacements previously proposed. A Swin Layer of the optimized model is implemented in C for profiling and to serve as the baseline for further comparison.

3.1 Accuracy Results of the AerialFormer-HW

An ablation study with the three hardware-friendly replacements was performed to evaluate their suitability and impact in performance. The three replacements evaluated were: HSwish or HardSwish4 (HSwish4) for GELU, Simple-BN or FFN-BN for LN, and SqrMax for SMax.

We also introduce a new HSwish variant, the HardSwish4 (HSwish4) function, as presented in equation 3.

$$\text{HardSwish4}(x) = x \cdot \frac{\text{ReLU4}(x + 2)}{4} \quad (3)$$

As observed in Figure 4, HSwish4 has a form function closer to the GELU than the standard HSwish, which suggests it as a better replacement. Also, it has a base-two denominator, which further simplifies its hardware implementation.

We propose HSwish4 as a novel hardware-friendly alternative to GELU for transformer models, and are not aware of prior work adopting this specific variant.

The results obtained with these experiments are compared in Table 2. The model (row 1) with the HSwish4 replacement of the GELU, direct BN replacement (Simple-BN) and SMax (not replaced) achieved the best results overall and was therefore selected to be implemented in hardware, being denominated AerialFormer-HW.

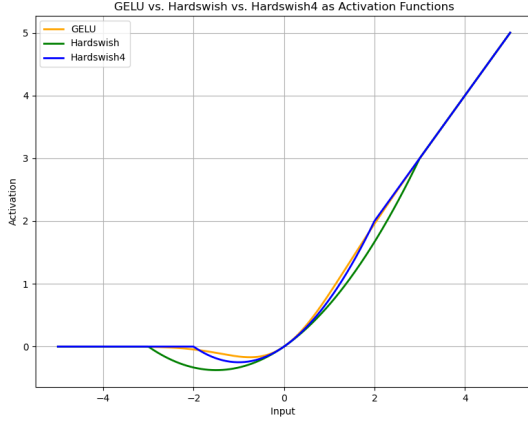


Figure 4: Comparison of GELU (yellow), HSwish (green) and HSwish4 (blue) activation functions

Table 2: Experimental Results for AerialFormer architectural modifications

Exp.	Norm.	Dist.	Act.	OA (%)	mIoU (%)
Base	LN	SMax	GELU	88.82	74.76
1	Simple-BN	SMax	HSwish4	88.84	74.70
(2)	LN	SMax	HSwish	88.75	74.52
(3)	Simple-BN	SMax	HSwish	88.62	74.26
(4)	FFN-BN	SMax	GELU	88.26	73.56
(5)	FFN-BN	SMax	HSwish	87.94	73.15
(6)	LN	SqrMax	GELU	87.34	71.90
(7)	LN	SqrMax	HSwish	87.03	71.42
(8)	FFN-BN	SqrMax	HSwish	84.54	67.17

Simple-BN replacements consistently outperformed the FFN-BN strategy across both metrics, as training starts from pre-trained Swin Transformer weights and Simple-BN introduces fewer architectural changes.

The SqrMax replacements (rows 6, 7 and 8) presented the poorest results, due to the loss of negative values which the pre-trained model expects.

3.2 Baseline Swin Layer in C

We developed a baseline embedded C application that implements the inference of the Swin Layer: WSA, general matrix multiplication, general matrix addition, bias addition, BN, HSwish4 and SMax. This modular approach provides a solid reconfigurable framework for further works and to be easily expanded.

The execution times shown in Table 3 were profiled on the embedded ARM Cortex-A53 processor, with compiler optimizations. The FFN takes the longest time to run, $\approx 58\%$, followed by the QKV Generation with $\approx 24\%$ and the WSA (excluding SMax) with $\approx 9\%$. These components are selected to be accelerated in hardware. The remaining 8.5% of the run time is composed of the padding, re-shaping, window extraction, de-padding logic, batch normalization

Table 3: Optimized (-O2) Software-Only time profiling

Sub Component	Floating-Point	
	Time (s)	Time (%)
QKV Gen.	5.1	23.9
WSA + Projection	2.0	9.4
FFN	12.4	58.2
Others	1.8	8.5
Total	21.3	100

and SMax, which will be kept in software for their relative small contribution to total execution time.

3.3 AerialFormer-HW Quantization

The AerialFormer-HW model was trained using a Quantization Aware Training (QAT) strategy [10, 18], quantizing the layers with different bitwidths for the activations (a) and weights (w). The performance of these quantization experiments is presented in Table 4.

Table 4: AerialFormer-HW’s performance for different quantizations

Model Quantization	OA (%)	mIoU (%)	Norm. FFN weight size (%)
Float32	88.8	74.7	100.0
(16,16)	84.2	66.5	50.0
(8,8)	82.2	63.7	25.0
(8,4)	65.3	40.9	25.0
(4,8)	64.9	39.5	12.5
(4,4)	63.9	38.6	12.5

(8,8) quantization provided the most promising result, reducing the weight size to 25% of the base floating-point-size at the cost of a reduction of the mIoU metric (63.7% vs. 74.7%). This 11% decay in mIoU is tolerable, especially because the clutter class (in red) is the most affected by quantization and the segmentation of critical semantic classes (buildings, vehicles, vegetation) is still mostly accurate, as compared in Figure 5.

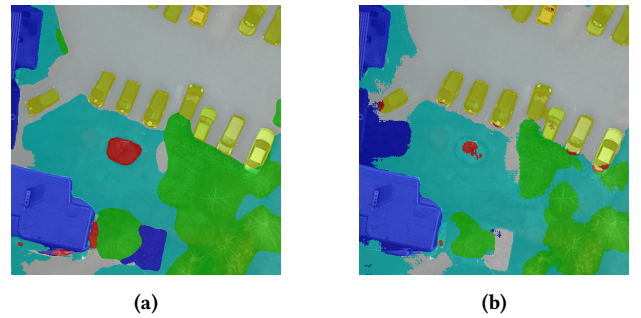


Figure 5: Comparison between Floating-Point (a) and (8,8) quantization (b) segmentation results

4 Swin Transformer Accelerators

Based on the profiling results in Table 3, the FFN, the QKV value generation, and the WSA were targeted for acceleration, effectively covering the most computationally intensive parts of the model. The accelerators were designed at the architectural level in C and synthesized using *Vitis HLS 2023.2*.

4.1 Dataflow-oriented Feed-Forward Network Accelerator

The FFN is mapped to a two-stage streaming pipeline, where the linear transformations and the activation function are fused to eliminate intermediate memory writes and buffering, as shown in Figure 6. Our design uses a dataflow strategy, such that all components work simultaneously, as soon as data is available to process.

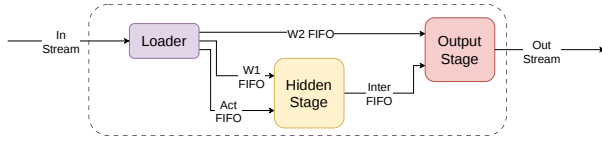


Figure 6: FFN Accelerator Topview

To be implemented efficiently as First-In First-Out (FIFOs), the streams in the dataflow architecture must follow the principle “one producer, one consumer” [25]. The Loader component ensures this, as it fans-out the data from the input stream to the respective component streams. Both input and output data streams, as well as the weights and inter stage FIFOs are kept 64-bit wide.

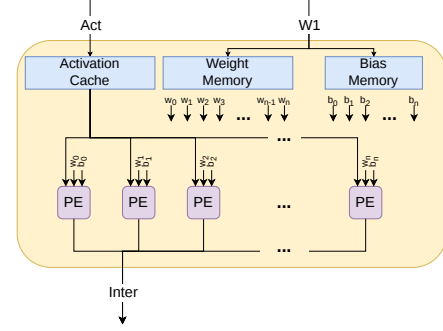
The hidden stage implements the first GEMM and the HSwish4 activation of the FFN, storing weights and biases locally to minimize external memory transfers. As shown in Figure 7, an array of 8 Processing Elements (PEs) computes multiple output elements in parallel, sharing activation inputs while using distinct weight and bias parameters.

The activation buffer caches one input row at a time from the FIFO interface, with 8-bit activations packed into 64-bit words delivering 8 elements per cycle to each PE. The PEs are comprised of 8 8-bit multipliers executing 8 Multiply And Accumulates (MACs) in parallel, an adder tree, an accumulator register and a HSwish4 unit, implemented with a multiplexer and a 25-bit multiplier, as shown in Figure 8.

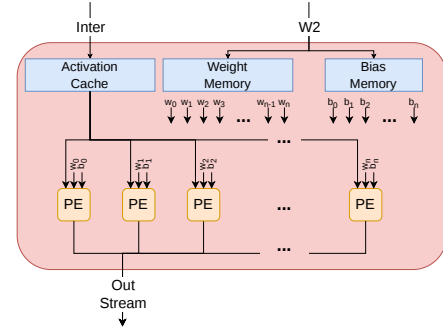
The output stage follows this same structure with its PEs having no HSwish4 unit, and their accumulator registers resized to accommodate the higher number of MACs.

4.2 QKV Generation and Projection Accelerator

The FFN accelerator architecture is reused to calculate the second-largest bottleneck: QKV value generation from the input. The IP is reconfigured to calculate the Query (Q), Key (K), and Value (V) matrix projections through a single GEMM operation therefore all three matrices are computed as a single, continuous block. After reading the weights to internal memory, the activations are read from external memory and streamed to the PEs. The resulting QKV block is streamed back to the CPU for reshaping for WSA computation.



(a) Hidden stage component



(b) Output stage component

Figure 7: FFN accelerator stages: hidden and output components

The FFN accelerator is also reconfigured and reused to compute the projection GEMM.

4.3 WSA Accelerator

The WSA Accelerator follows the same structure of the FFN accelerator, with 64 MACs in parallel. The key difference between both accelerators lies from where the data is read. In the WSA Accelerator both the input and weight matrices are kept in the on-chip memory, as the data needed to complete the computation is much smaller.

Bundles of 64 Query and Key windows are streamed to the accelerator, where their respective multiplications are computed. The result of the QK window multiplications are streamed back to the CPU to apply SMax. After all QK windows are generated the IP is reused to compute the attention result windows by multiplying with the partitioned V windows.

5 ARES Hardware/Software System

The final Hardware/Software implementation was implemented with *Vivado 2023.2* and tested in a Zynq Ultrascale+ SoC-FPGA device, with the hardware resource consumption of the implementation reported in Table 5. As expected, most of the resources are used by the FFN and the Attention accelerator, with 80% of the

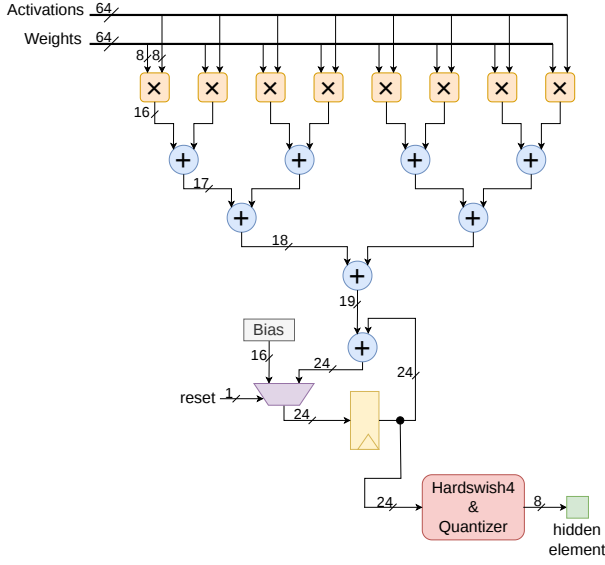


Figure 8: PE for hidden element generation

total used LUTs and 88% of the total used Block Random-access Memories (BRAMs) being assigned to this component. Only the final 25-bit multipliers used by the HSwish4 activation are mapped to DSPs, all other 8-bit multipliers are mapped to LUTs.

Table 5: Implemented design resource usage on the XCZU7EV device

	LUT (230,400)	FF (460,800)	DSP (1,728)	BRAM_36K (312)
FFN Accelerator	12,476	3,561	8	33
WSA Accelerator	8,739	2,404	0	50
DMA 0 & 1	2,948	4,434	0	6
Connect	3,114	4,668	0	0
PS Reset System	14	33	0	0
Total	27,291	15,100	8	92

The final version of the ARES accelerator was clocked at 150 MHz, limited by the critical path of its adder tree. The time profiling of the ARES hardware/software system is presented in Table 6. After hardware acceleration of the main components, the bottleneck became the “Others” component (data reshape, Quantization and De-quantization, BN and SMax routines) which in the software-only baseline accounted for only 8.5% of execution time.

The ARES system speeds up the execution of the end-to-end Swin Layer in about 10.14×, with a FFN speed-up of 206× when compared to the baseline. This is consistent with Amdahl’s Law, as the non-accelerated portion of the workload ultimately limits the achievable system-level speedup.

We estimated the ARES system power using *Vivado 2023.2 Power Analysis* tool with the post-implementation data, obtaining a estimation of 4.9 W at 150 MHz for the complete hardware/software ARES system, with 2.5 W attributed to the ARM Cortex-A53 processor.

Table 6: ARES Accelerator Time Profiling

Sub Component	Time (s)	Time (%)	Speedup
QKV Gen.	0.05	≈2	102×
WSA + Projection	0.03	≈2	67×
FFN Accel.	0.06	≈3	206×
Others.	1.96	93.3	-
Total	2.1	100	10×

6 Conclusions

This work presented ARES, a complete hardware-software co-design framework for accelerating Swin Transformer layers on embedded SoC-FPGA platforms. By profiling the AerialFormer-Tiny model and targeting its dominant computational kernels, we developed dataflow-oriented accelerators for the Feed-Forward Network (FFN), QKV projection, and Window Self-Attention (WSA), covering over 90% of the original execution time.

Key innovations include hardware-friendly replacements for expensive transformer operations, namely HSwish4 for GELU and Simple-BN for LN, which preserve competitive segmentation accuracy (74.7% mIoU on Potsdam) under 8-bit quantization. The parameterized PE-array architecture enables flexible resource-performance trade-offs, while the streaming dataflow eliminates intermediate buffering and maximizes parallelism in the FFN accelerator, achieving 206× speedup for this component alone.

Executing on a Zynq Ultrascale+ SoC-FPGA device, ARES delivers 10.14× end-to-end acceleration of a Swin layer versus ARM-only execution, with a total power consumption of 4.9 W. The total Swin Layer execution approaches the Amdahl’s law speedup limit bounded by the execution time of the small software components. These results demonstrate the feasibility of transformer-based aerial image segmentation on resource-constrained edge devices, enabling real-time applications such as disaster response and urban monitoring.

Future work will extend the framework to full encoder-decoder stacks by optimizing control logic for variable layer sizes, accelerating transposed convolutions in the decoder, and refining quantization-aware training to further minimize accuracy degradation. Additional opportunities include replacing SoftMax (SMax) with Square-Max (SqrMax) through targeted fine-tuning and porting the design to lower-power platforms.

Acknowledgments

This work is funded by national funds through FCT — Fundação para a Ciência e a Tecnologia, I.P., under projects UID/6486/2025 (<https://doi.org/10.54499/UID/6486/2025>), UID/PRR/6486/2025 (<https://doi.org/10.54499/UID/PRR/6486/2025>), UID/PRR2/6486/2025 (<https://doi.org/10.54499/UID/PRR2/6486/2025>), UID/50021/2025 (DOI: <https://doi.org/10.54499/UID/50021/2025>), UID/PRR/50021/2025 (DOI: <https://doi.org/10.54499/UID/PRR/50021/2025>), and under project 2023.16747.ICDT (SMARTChip, DOI: <https://doi.org/10.54499/2023.16747.ICDT>).

João Barreiros Rodrigues acknowledges FCT for the support through grant BI|2025/772– Proj. SMARTChip – Ref^a LISBOA2030-FEDER-00692100.

References

- [1] Jimmy Lei Ba. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [2] Jason Cong, Bingjun Xiao, Peng Zhang, Dong Liang, Yong Zhao, and Jiajie Liu. 2018. When Deep Learning Meets FPGA: A Survey. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 12, 2 (2018), 1–41. doi:10.1145/3289185
- [3] Qiwei Dong, Xiaoru Xie, and Zhongfeng Wang. 2024. Swat: An efficient swin transformer accelerator based on fpga. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 515–520.
- [4] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [5] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).
- [6] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*. 1314–1324.
- [7] Meng-Hsun Hsieh, Xuan-Hong Li, Yu-Hsiang Huang, Pei-Hsuan Kuo, and Juin-Dar Huang. 2024. A Hardware-Friendly Alternative to Softmax Function and Its Efficient VLSI Implementation for Deep Learning Applications. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [8] Ruofei Hu, Binren Tian, Shouyi Yin, and Shaojun Wei. 2018. Efficient hardware architecture of softmax layer in deep neural network. In *2018 IEEE 23rd International Conference on Digital Signal Processing (DSP)*. IEEE, 1–5.
- [9] ISPRS. 2018. 2D Semantic Labeling Contest – Potsdam. <https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-potsdam.aspx>. Working Group III/4, International Society for Photogrammetry and Remote Sensing. Accessed: 2026-04-23.
- [10] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2704–2713.
- [11] Wonkyung Jung, Daejin Jung, Byeongho Kim, Sunjung Lee, Wonjong Rhee, and Jung Ho Ahn. 2019. Restructuring batch normalization to accelerate CNN training. *Proceedings of machine learning and systems* 1 (2019), 14–26.
- [12] Beom Jin Kang, Hae In Lee, Seok Kyu Yoon, Young Chan Kim, Sang Beom Jeong, Hyun Kim, et al. 2024. A survey of FPGA and ASIC designs for transformer inference acceleration and optimization. *Journal of Systems Architecture* (2024), 103247.
- [13] Yun Liang, Kyle Rupnow, Yanan Li, Dongbo Min, Minh N Do, and Deming Chen. 2012. High-Level Synthesis: Productivity, Performance, and Software Constraints. *Journal of Electrical and Computer Engineering* 2012, 1 (2012), 649057.
- [14] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.
- [15] Zhiyang Liu, Pengyu Yin, and Zhenhua Ren. 2023. An efficient fpga-based accelerator for swin transformer. *arXiv preprint arXiv:2308.13922* (2023).
- [16] Shashank Nag, Gourav Datta, Souvik Kundu, Nitin Chandrachoodan, and Peter A Beerel. 2023. ViTA: A vision transformer inference accelerator for edge applications. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [17] Razvan Nane, Vlad-Mihai Sima, Christian Pilato, Jongsok Choi, Blair Fort, Andrew Canis, Yu Ting Chen, Hsuan Hsiao, Stephen Brown, Fabrizio Ferrandi, et al. 2015. A survey and evaluation of FPGA high-level synthesis tools. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 10 (2015), 1591–1604.
- [18] Alessandro Pappalardo and contributors. 2025. Brevitas: PyTorch Quantization-Aware Training Library. <https://github.com/Xilinx/brevitas>. Accessed: 2025-10-24.
- [19] Mahadev Satyanarayanan. 2017. The Emergence of Edge Computing. *Computer* 50, 1 (2017), 30–39. doi:10.1109/MC.2017.9
- [20] Sheng Shen, Zhewei Yao, Amir Gholami, Michael Mahoney, and Kurt Keutzer. 2020. Povernorm: Rethinking batch normalization in transformers. In *International conference on machine learning*. PMLR, 8741–8751.
- [21] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal* 3, 5 (2016), 637–646. doi:10.1109/JIOT.2016.2579198
- [22] Tomislav Sledovic. 2019. Adaptation of convolution and batch normalization layer for CNN implementation on FPGA. In *2019 Open Conference of Electrical, Electronic and Information Sciences (eStream)*. IEEE, 1–4.
- [23] Ahram Song and Yongil Kim. 2020. Semantic segmentation of remote-sensing imagery using heterogeneous big data: International society for photogrammetry and remote sensing potsdam and cityscape datasets. *ISPRS International Journal of Geo-Information* 9, 10 (2020), 601.
- [24] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [25] Mário Véstias, Paulo Flores, and Horácio Cláudio de Campos Neto. 2025. Síntese de alto nível em FPGA. (2025).
- [26] Kashu Yamazaki, Taisei Hanyu, Minh Tran, Adrian Garcia, Anh Tran, Roy McCann, Haitao Liao, Chase Rainwater, Meredith Adkins, Andrew Molthan, et al. 2023. AerialFormer: Multi-resolution transformer for aerial image segmentation. *arXiv preprint arXiv:2306.06842* (2023).
- [27] Zhuliang Yao, Yue Cao, Yutong Lin, Ze Liu, Zheng Zhang, and Han Hu. 2021. Leveraging batch normalization for vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 413–422.